COMP 3613 - Assignment 2

Test Planning, Implementation and Reporting

Student Awards (incentive system)

Name	Benjamin Pfister
Student ID	816037378
Repository	https://github.com/BenPfister03/student-awards
Date	2026-11-10

Contents

1. Project description
2. Use case diagram
3. Entity relationship diagram
4. Application wireframes
5. Test plans - UAT, unit, integration (+ traceability)
6. Implementation - repository, unit, integration and API tests
7. Reports - unit, integration, Postman, UAT, defects

1. Project description

Student Awards is a FastAPI / SQLModel web application that turns volunteering into an incentive scheme. Students log volunteer hours against an activity with a short proof note; an administrator reviews the pending queue and approves or rejects each entry (rejections need a reason). Approved hours add up to the student's total, place them on a leaderboard ranked by approved hours, and unlock milestone badges when thresholds (Bronze 10h, Silver 25h, Gold 50h) are crossed. Students can then redeem their balance (approved hours minus hours already spent) for prizes with limited stock; the administrator marks each redemption as fulfilled.

Workflow	Actors	Done when
1. Log and approve volunteer hours	Student, Admin	Entry shows approved/rejected and the student's total hours update on approval.
2. Leaderboard and milestones	Student	Rank and unlocked milestones reflect the latest approved hours.
3. Redeem a prize	Student, Admin	Balance drops and the redemption shows fulfilled.

Architecture. Layered: routers (HTML views + /api) -> services (business rules) -> repositories -> SQLModel tables. Authentication is a JWT held in an HTTP-only cookie; authorisation is decided from the user's database role through the *AdminDep* dependency. SQLite is used locally and in tests; Postgres in production.

2. Use case diagram

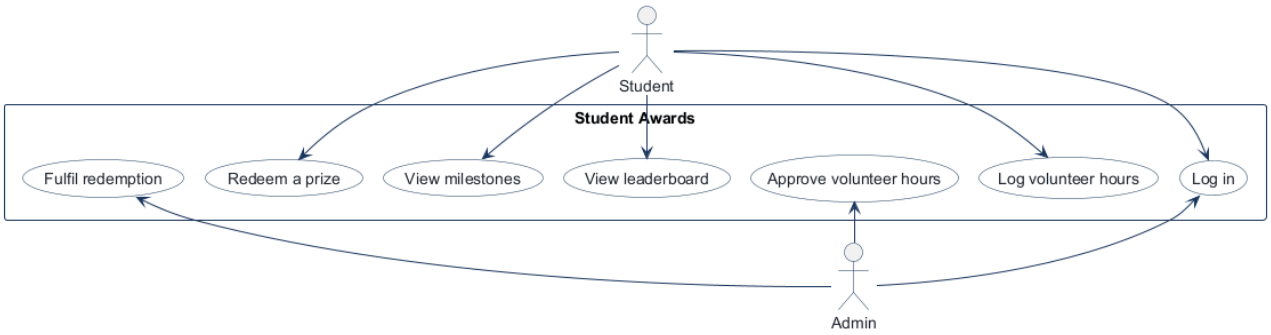


Figure 1 - Student and Admin actors; *Log in* is shared. Source: docs/diagrams/use-case.json

3. Entity relationship diagram

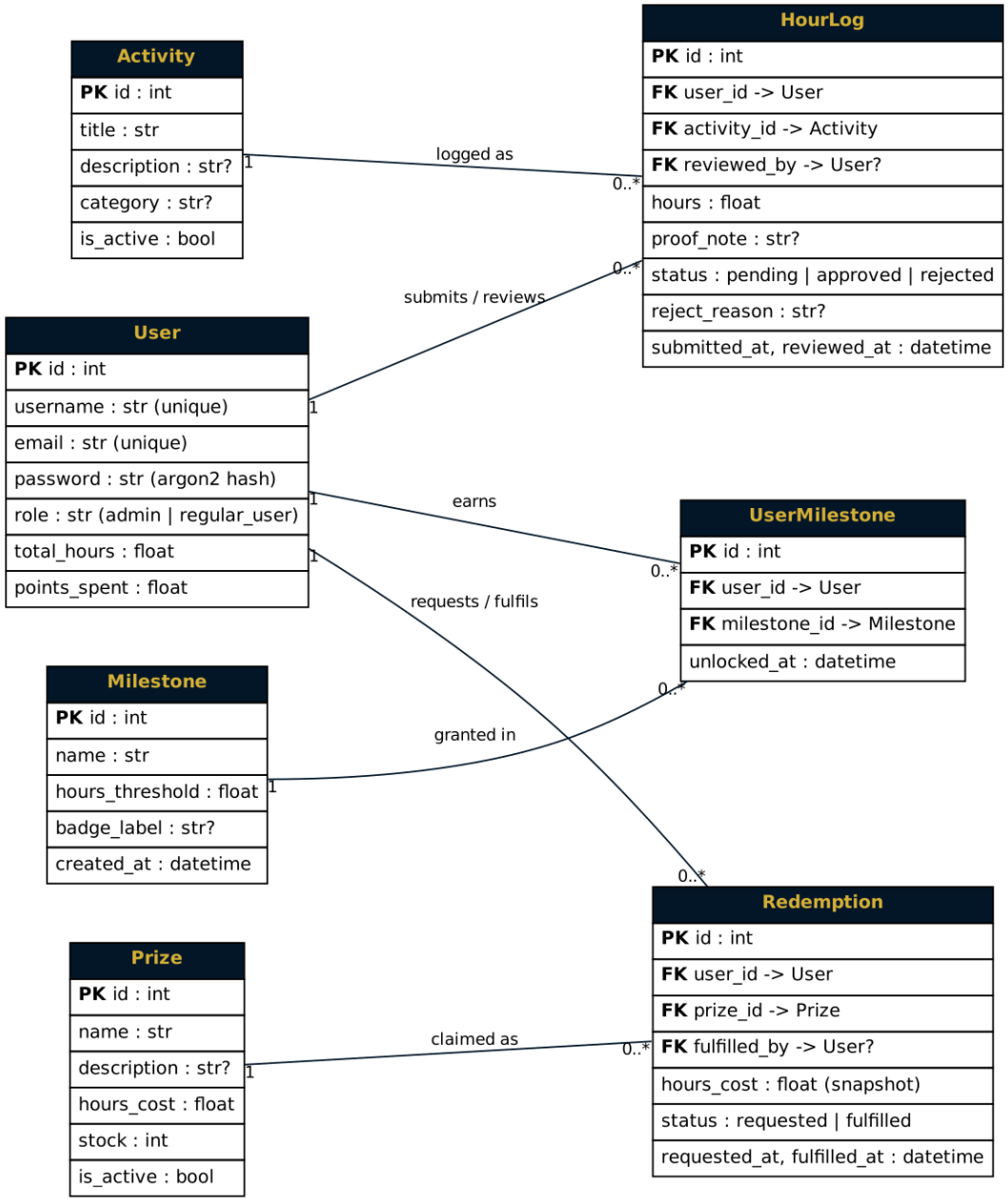
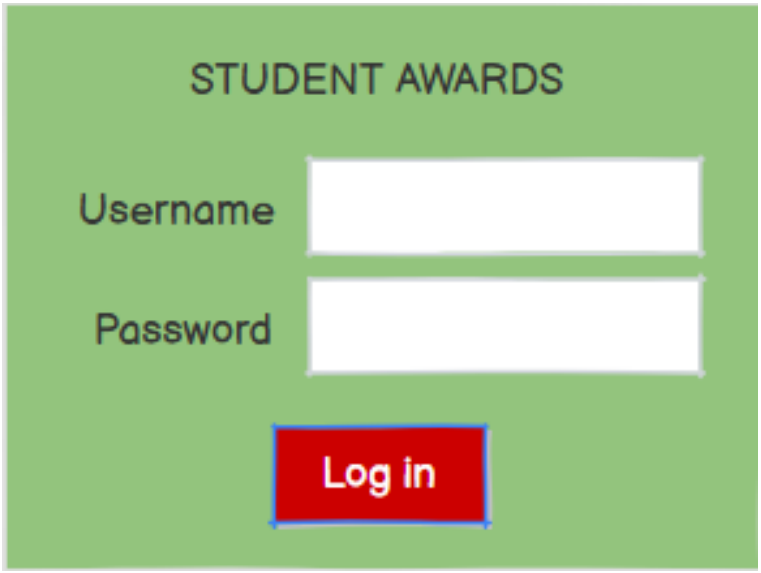


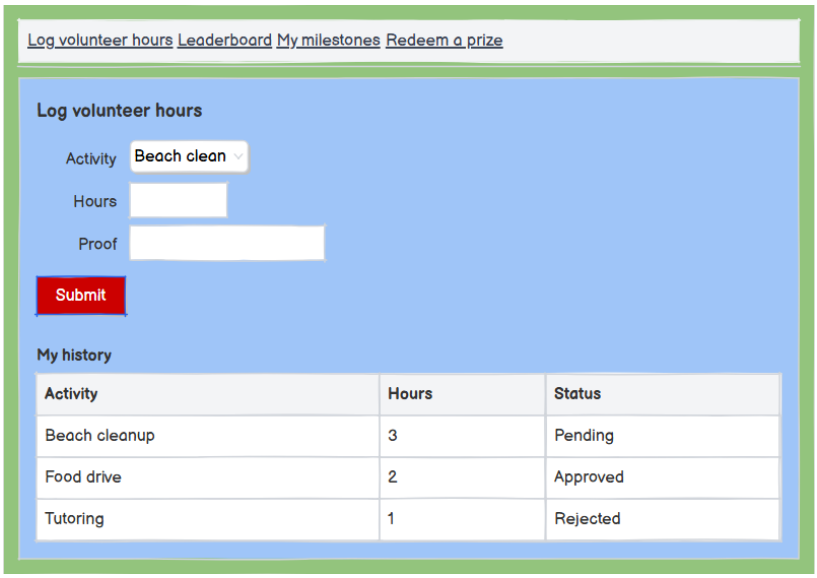
Figure 2 - ERD generated from the SQLModel classes in app/models (docs/diagrams/erd.dot). Both HourLog.reviewed_by and Redemption.fulfilled_by are nullable foreign keys to User (the admin).

4. Application wireframes

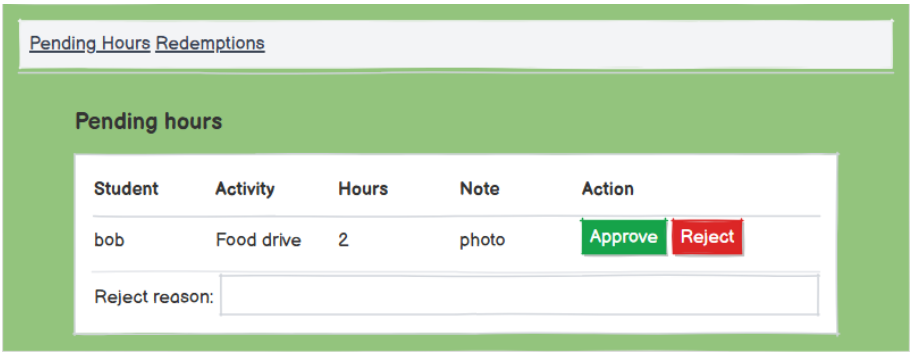
1. Log in



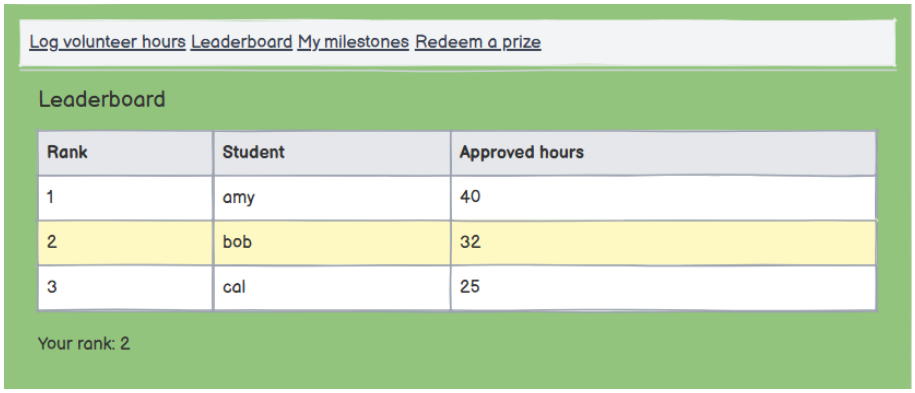
2. Log volunteer hours Student



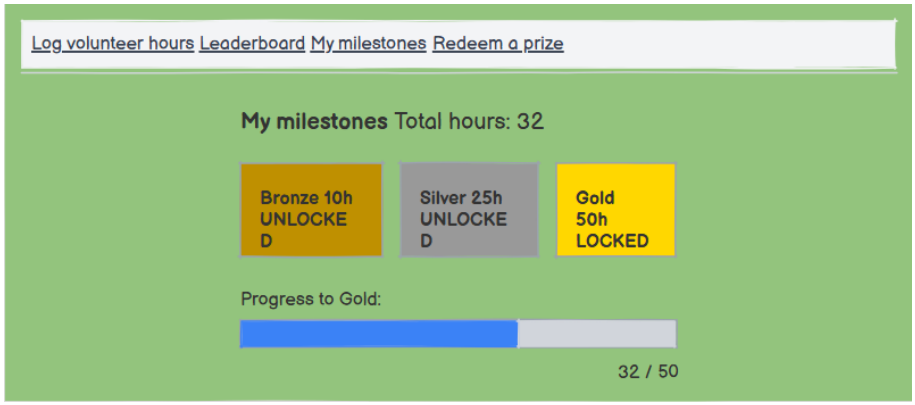
3. Approve volunteer hours Admin



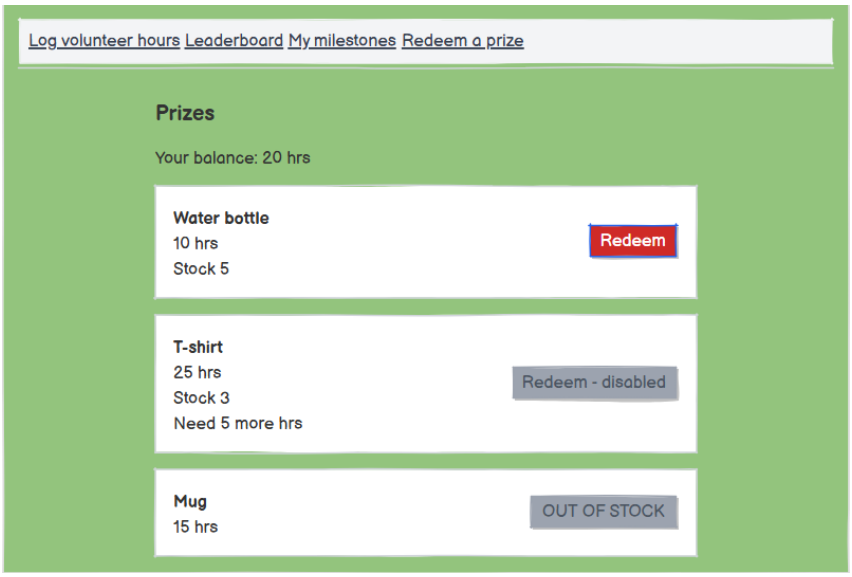
4. View leaderboard



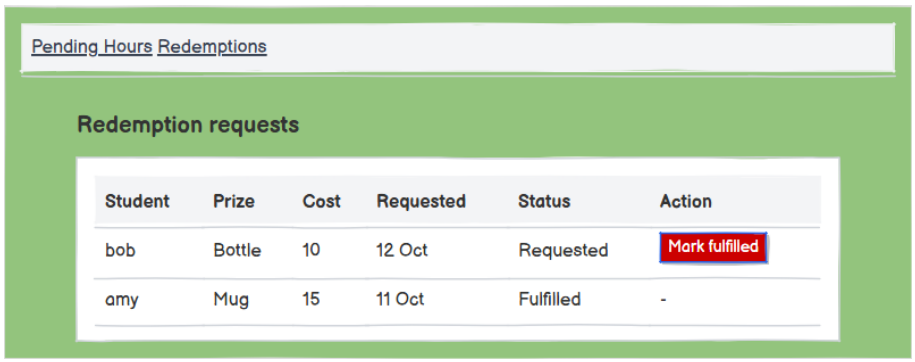
5. View milestones



6. Redeem a prize



7. Fulfil redemption Admin



5. Test plans

5.1 Strategy

Level	Goal	Tooling	Isolation / data
Unit	Verify each module in isolation: utilities, models/schemas, repository, services, auth guard.	pytest (tests/unit)	Private in-memory SQLite per test (StaticPool); no HTTP, no app engine; factory helpers in tests/conftest.py.
Integration	Verify the layers working together through real HTTP: routing, cookies, authorisation, services, DB, templates.	pytest + FastAPI TestClient (tests/integration)	Temporary SQLite file, tables dropped and recreated before each test; fixtures build admin, bob, amy, cal and the catalogue.
API	Black-box checks against the running server including redirects and cookie sessions.	Postman collection (postman/)	Run against a freshly seeded DB (python manage.py init).
UAT	Real user completes tasks from the wireframes and judges the success criteria.	Manual, browser	Seeded demo data; script in 5.3.

Entry criteria: dependencies installed, *.env* present. **Exit criteria:** 100% of unit and integration tests pass (known-defect tests may report *xfailed*), the Postman run has 0 failed assertions, every UAT row is Pass or has a logged defect. **Out of scope:** performance, browser compatibility, deployment smoke tests.

5.2 Traceability matrix

Workflow / feature	UAT	Integration	Postman folder	Main unit modules
Log in / out, register, tokens	01-03, 17	IT-AUTH	1	UT-SEC, UT-AUT, UT-REP
Authorisation (roles)	02, 16	IT-RBAC	6	UT-DEP
WF1 Log and approve hours	04-09	IT-WF1	2, 3	UT-HRS, UT-MOD
WF2 Leaderboard and milestones	10, 11, 15	IT-WF2	4	UT-HRS (milestone awarding)
WF3 Redeem and fulfil prize	12-15	IT-WF3	5	UT-RED
JSON API / contract	-	IT-API	0	UT-MOD (UserResponse)
Known defects	-	IT-DEF	-	-

5.3 User Acceptance Test plan

Run *python manage.py init* then *python manage.py run* and open the app in a browser. The plan below summarizes the key acceptance scenarios and is intentionally shorter than the detailed execution report in section 7.4.

Tester Name	Benjamin Pfister
Tester Position	Student / Project Developer
Date of Test (DD/MM/YYYY)	10/11/2026

Test Case	Pre-conditions	Test Steps	Test Criteria	Success
1.1 Test Login	User is signed out.	1. Open /login. 2. Log in as bob/bobpass. 3. Review the dashboard.	The user is authenticated and redirected to the student home page with summary cards visible.	Pass / Fail
1.2 Test Log Hours	Student is signed in and an active activity exists.	1. Open Log Hours. 2. Submit a valid 10h entry with proof note. 3. Review history.	The entry is listed as pending and is associated with the correct student.	Pass / Fail
1.3 Test Approve Hours	Admin account exists and pending entries exist.	1. Sign in as admin. 2. Open Pending Hours. 3. Approve and reject entries.	Approved hours update the student's total and rejected entries show a reason.	Pass / Fail
2.1 Test Leaderboard and Milestones	Student has approved hours and milestone thresholds.	1. Open Leaderboard. 2. Open Milestones. 3. Verify rank and milestone status.	The rank order and milestone state reflect the approved-hour totals correctly.	Pass / Fail
2.2 Test Prize Redemption	Student has awarded hours and at least one prize is affordable.	1. Open Prizes. 2. Redeem an affordable prize. 3. Sign in as admin and fulfil it.	Balance, stock, redemption status, and fulfilment all update correctly.	Pass / Fail
2.3 Test Access Control and Logout	A student is signed in and an admin-only route is available.	1. Attempt to access an admin route as bob. 2. Click Logout. 3. Retry a protected page.	The user is blocked from admin routes and is logged out correctly.	Pass / Fail

Wireframe vs. implementation - confirm during UAT (from template review): (a) wireframe 5 shows progress toward Gold; the app shows progress toward the *next* locked milestone. (b) wireframe 2's history has Activity/Hours/Status; the app adds Reject reason and Submitted columns. (c) wireframe 3 shows one Reject-reason box; the app has one per row.

5.4 Unit Test Plan

Unit Test	Input	Expected Output
test_new_user()	"bob", "bobpass", "bob@mail.com"	newuser.username == "bob" newuser.email == "bob@mail.com"
test_toJSON()	"bob", "bobpass"	{ id: None, "username": "bob" }
test_hashed_password()	"mypass"	newuser.password != "mypass"
test_check_password()	"mypass"	user.check_password("mypass") == True

5.5 Integration Test Plan

Test	Dependencies	Description
test_create_user()	create_user()	Ensures user record in database has the correct values.
test_authenticate()	create_user(), authenticate()	Ensures authenticate() returns true when given the correct credentials.
test_get_all_users_json()	create_user(), get_all_users_json()	Verifies JSON data of all users in the system.
test_update_user()	create_user(), update_user(), get_user()	Ensures an updated user is retrieved with the updated values.

6. Implementation

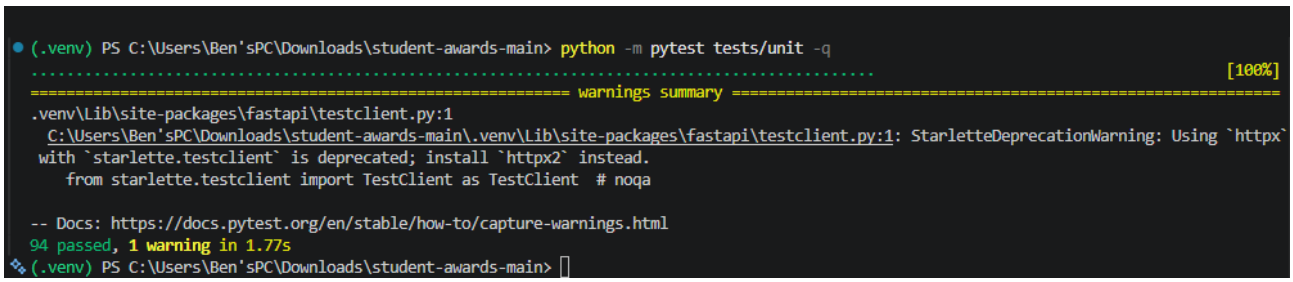
Repository: https://github.com/BenPfister03/student-awards

Deliverable	Location in repo	How to run
Unit tests	tests/unit/ (8 files, 94 cases)	*pytest tests/unit -v*
Integration tests	tests/integration/ (7 files, 122 cases)	*pytest tests/integration -v*
Shared fixtures	tests/conftest.py	temp DB, user/catalogue factories, login factory
API tests (Postman, exported JSON)	postman/student-awards.postman_collection.json (63 requests, 163 assertions)	Collection Runner, or *newman run postman/student-awards.postman_collection.json -e postman/local.postman_environment.json*
pytest configuration	pyproject.toml [tool.pytest.ini_options]	testpaths, pythonpath, markers (unit / integration)
Test documentation	TESTING.md	-

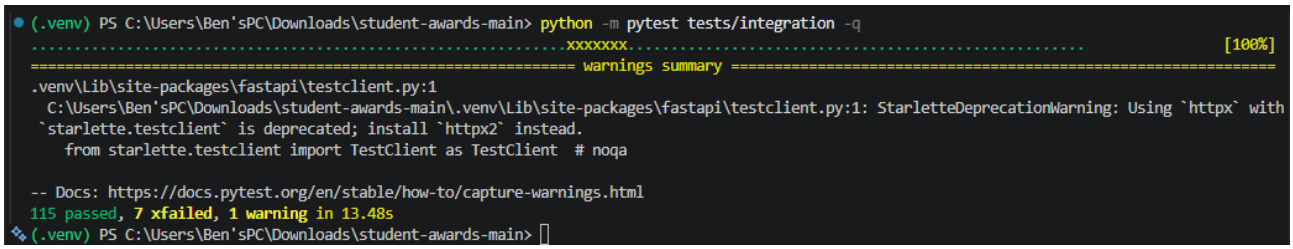
The test database is a temporary SQLite file created by tests/conftest.py *before* the app is imported, so running the suite never touches database.db. Run both suites together with *pytest*.

7. Reports

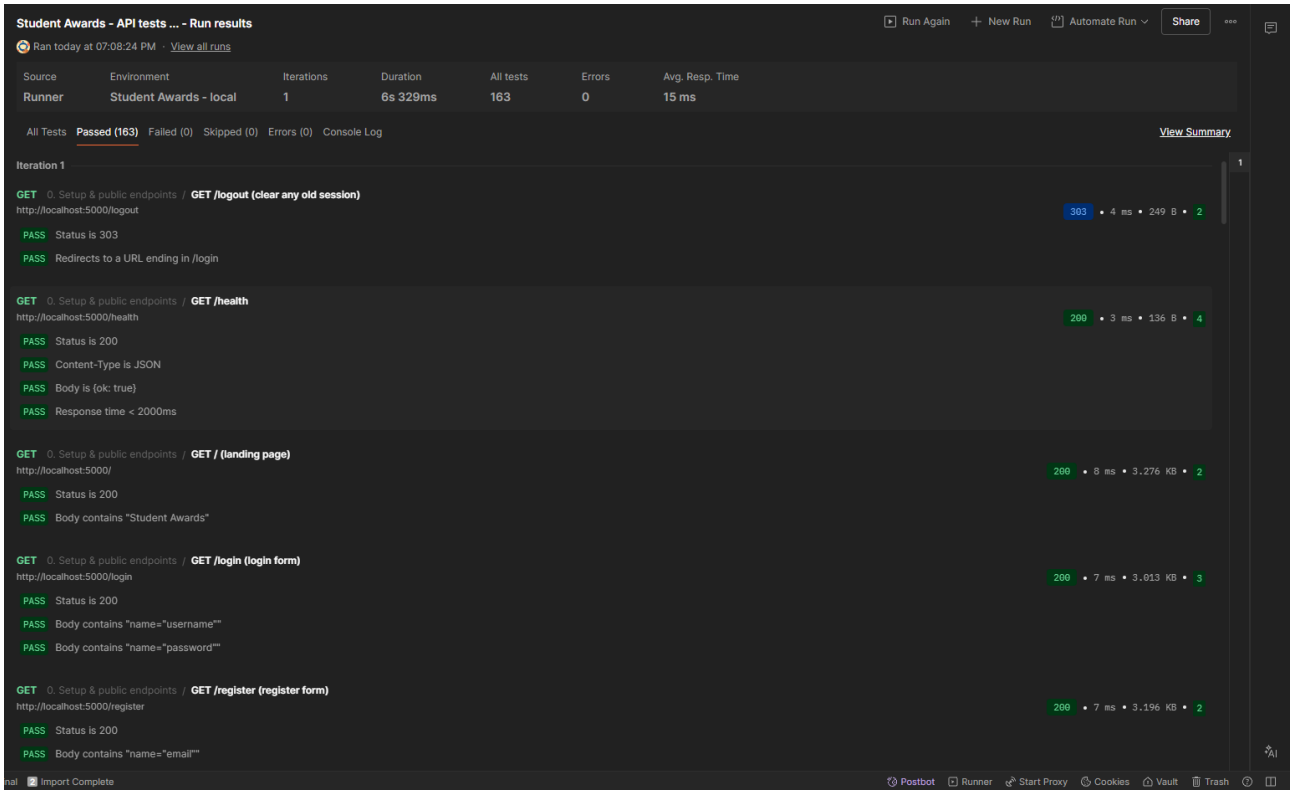
7.1 Unit tests report



7.2 Integration tests report



7.3 Postman collection run report



7.4 User Acceptance Test report

Tester Name	Tester Position	Date of Test (DD/MM/YYYY)
Benjamin Pfister	Student / Project Developer	10/11/2026

Test Case	Pre-conditions	Test Steps	Test Criteria	Success (indicate with a ✓)
1.1 Test Login	User is signed out.	Open /login, sign in as bob / bobpass, then review the home page.	The user is authenticated, the dashboard loads, and summary cards are visible.	✓
1.2 Test Log Hours	Student is signed in and an active activity exists.	Open Log Hours, enter a valid 10h entry with proof note, then submit.	The activity is accepted as pending and appears in the student's history.	✓
1.3 Test Approve Hours	Admin is signed in with a pending entry to approve.	Open Pending Hours, review the submitted hours, and approve the request.	The entry is accepted, approved hours change, and the student total updates.	✓
2.1 Test Leaderboard and Milestones	Student has approved hours and leaderboard data exists.	Open the leaderboard and review rank ordering.	The leaderboard is sorted correctly by approved hours and the user's rank is shown.	✓
2.2 Test Prize Redemption	Student has approved hours and a prize is affordable.	Open Prizes, choose an affordable item, and redeem it.	The prize request is created and the student's remaining balance is reduced correctly.	✓
2.3 Test Access Control and Logout	Student is signed in and admin-only routes exist.	Open an admin path such as /admin/approvals while signed in as bob, then log out and retry a protected route.	The unauthorized page is shown, admin data remains hidden, and the user is correctly signed out.	✓

General Comments	Accepted | Rejected	Signature
All core workflows passed during manual user acceptance testing.	Accepted	Benjamin Pfister